

Pontifícia Universidade Católica de Minas Gerais
Instituto de Ciências Exatas e Informática
Algoritmos e Estruturas de Dados II

Trabalho Prático IV

Regras Básicas

1. extends Trabalho Prático 03
2. Neste trabalho, você deverá utilizar o Dataset fornecido no Trabalho Prático 02 como base para os exercícios.

Ordenação Parcial

1.  **Ordenação PARCIAL por Seleção em C**: Refaça a Questão "Ordenação por Seleção" do TP02 considerando a ordenação parcial com k igual a 10.
2.  **Ordenação PARCIAL por Inserção em Java**: Refaça a Questão "Ordenação por Inserção" do TP02 considerando a ordenação parcial com k igual a 10.
3.  **Ordenação PARCIAL por Quicksort em C**: Refaça a Questão "Ordenação por Quicksort" do TP03 considerando a ordenação parcial com k igual a 10.
4.  **Ordenação PARCIAL por Heapsort em Java**: Refaça a Questão "Heapsort" do TP03 considerando a ordenação parcial com k igual a 10.

Árvores Binárias

5.  **Árvore Binária em Java**: Crie uma Árvore Binária, fazendo inserção de registros conforme a entrada padrão. A chave de pesquisa é o atributo **modelo**. Não insira um elemento se sua chave estiver na árvore. Em seguida, pesquise se alguns registros estão na árvore, mostrando seus respectivos caminhos de pesquisa. A entrada padrão possui o mesmo formato da questão

"Pesquisa Binária" do TP02. Os registros referentes à primeira parte da entrada devem ser inseridos na árvore. A segunda parte da entrada contém as chaves a serem pesquisadas.

A saída padrão é composta por várias linhas, uma para cada pesquisa. Cada linha é composta pelo caminho de ponteiros (`raiz, esq ou dir`) utilizados na pesquisa e, no final, pelas palavras SIM ou NAO, indicando o resultado da pesquisa. Ao final, os registros presentes na árvore devem ser apresentados usando caminhamento em ordem.

6.  **Árvore Binária em C**: Repita o exercício anterior "Árvore Binária", realizando a implementação em C.

Árvores Balanceadas

7.  **Árvore AVL em Java**: Refaça a questão "Árvore Binária em Java" com Árvore AVL.
8.  **Árvore Bicolor em C**: Refaça a questão "Árvore Binária em C" com Árvore Bicolor.

Tabelas Hash

9.  **Tabela Hash Direta com Reserva em Java**: Refaça a questão "Árvore Binária em Java" com Tabela *Hash* Direta com Reserva. A função de transformação será **(ASCII modelo) mod tamTab**, onde **(ASCII modelo)** é a soma dos códigos ASCII do atributo **modelo** e **tamTab** é o tamanho da tabela. Considere que a tabela tenha tamanho 31 e a área de reserva tamanho 19. Durante a inserção, caso a inserção do veículo não possa ser realizada por causa de colisão e falta de espaço na área de reserva, imprima o modelo do veículo. A saída padrão será a posição de cada elemento procurado na tabela (na *hash* ou na área de reserva, isto é, de 0 a 49), seguido do veículo formatado. Se o elemento procurado não estiver na tabela, escreva -1.
10.  **Tabela Hash Direta com Rehash em C**: Refaça a questão "Tabela Hash Direta com Reserva em C" com Tabela *Hash* Direta com *Rehash*. A primeira função de transformação será **(ASCII modelo) mod tamTab**, onde **tamTab** (tamanho da tabela) é 83. A segunda função de transformação será **(ASCII modelo + 1) mod tamTab**.
11.  **Tabela Hash Indireta com Lista Simples em Java**: Refaça a questão "Tabela Hash Direta com Reserva em C" com Tabela *Hash* Indireta com Lista Simples. A função de transformação será **(ASCII modelo) mod tamTab**, onde **tamTab** (tamanho da tabela) é 31.